Smart AgriTech EMS Documentation

SmartAgriTech EMS — Optimization & Scaling Guide

Version: 4.0 · **Last updated:** June 2026
Stack: Node.js + Express + Prisma + PostgreSQL + TimescaleDB + Redis (optional) + Socket.IO · Flutter (`app/`)

Complete production documentation for all **60** performance, scalability, and correctness optimizations (`P-01` through `P-60`) on the SmartAgriTech EMS platform. Every item is ✅ **Fixed** in the v3.1 implementation branch reflected in this repository.

> **How to read this document**
>
> - Each item has a stable ID (`P-xx`), severity, category, location, phase, problem analysis, impact, production fix, files changed, and verification steps.
> - All items are marked ✅ **Fixed** — there are no Partial or Pending statuses in v4.0.
> - Redis-dependent features degrade gracefully when `REDIS_URL` is unset (see Section 1).

Table of Contents

---

0. Implementation Summary

Status	Count	Meaning
✅ Fixed	60	Fully implemented and verified in codebase
🟡 Partial	0	—
⏳ Pending	0	—

By phase (all complete)

Phase	Scope	Status
Phase 0	Client correctness, anomaly N+1	✅ Complete
Phase 1	Indexes, ingest SQL, aggregates, auth cache, rate limits, client fixes	✅ Complete
Phase 2	Redis, BullMQ, Socket cluster, PgBouncer, per-device ingest keys	✅ Complete
Phase 3	TimescaleDB, retention, observability, archival, JWT refresh, actuation ack	✅ Complete

v3.1 headline improvements (closed all remaining gaps)

Area	Key deliverables
Write path	Bulk `unnest` SQL (P-06), BullMQ batch worker + `createMany` (P-08/P-10), Redis latest + 60s flush (P-09), `sensor_reading_values` narrow table (P-14)
Queues	`ingest`, `anomaly`, `email`, `device-delete` workers in `jobQueues.js` (P-39/P-40/P-41/P-55/P-57)
Security	Per-device ingest keys (`ingestAuth.js`), JWT 15m + refresh 30d (P-47/P-49)
IoT actuation	`DeviceCommand` model, `PATCH /devices/:id/switch`, `POST /api/ingest/command-ack` (P-59)
Read path	`sensor_readings_hourly` for ranges >7d, `DATABASE_READ_URL` read client (P-32/P-36)
Ops	`/metrics`, structured logging, `pgbouncer.ini`, `archive-cold-data.js` (P-30/P-50/P-38)
Flutter	`com.smartagritech.ems`, pagination everywhere, silent token refresh (P-44/P-48)

DB migration: run `scripts/migrate-v3.1.sql` when `prisma db push` fails on Timescale hypertable PK constraints.

---

1. Production Prerequisites

Environment variables (`ems/ems-backend/.env`)

See `ems/ems-backend/.env.example` for the full template. Minimum production set:

```
DATABASE_URL=postgresql://postgres:PASSWORD@localhost:5432/ems
JWT_SECRET=<strong-secret>
JWT_EXPIRES_IN=15m
JWT_REFRESH_DAYS=30
INGEST_API_KEY=<global-fallback-gateway-secret>
CLIENT_URL=http://localhost:3000,http://10.0.2.2:3000

# Strongly recommended (Phase 2+ features):
REDIS_URL=redis://localhost:6379
INGEST_WORKER_CONCURRENCY=4
INGEST_BATCH_MAX=50
INGEST_BATCH_MS=100
INGEST_DEVICE_MAX_PER_MIN=120
SKIP_PG_CURRENT_VALUE=true
VALUE_FLUSH_MS=60000
REF_CACHE_TTL_SEC=300

# Pool tuning (P-31) — or point DATABASE_URL at PgBouncer port 6432 (P-30)
DB_POOL_MAX=20
DB_POOL_IDLE_MS=30000
DB_POOL_TIMEOUT_MS=10000

# Optional read replica for dashboard/analytics (P-32)
# DATABASE_READ_URL=postgresql://USER:PASSWORD@replica:5432/ems

# Optional SES/custom SMTP (P-55)
# SMTP_HOST=email-smtp.us-east-1.amazonaws.com
# SMTP_PORT=587
# SMTP_USER= SMTP_PASS= EMAIL_FROM=
```

Variable	Required	Enables
`DATABASE_URL`	Yes	Prisma + TimescaleDB (via PgBouncer in prod)
`REDIS_URL`	No	BullMQ queues, L1/L2 cache, cluster rate limits, Socket.IO adapter, Redis user cache
`DATABASE_READ_URL`	No	Read replica client for aggregates and browse endpoints

Variable	Required	Enables
`SKIP_PG_CURRENT_VALUE`	No	When `true` (default with Redis), skip Postgres `currentValue` on ingest hot path
`INGEST_BATCH_*`	No	BullMQ micro-batch size and flush interval

TimescaleDB (installed)

- **Extension:** `timescaledb` **2.27.2** on PostgreSQL **17.x**
- **Install script:** `ems/ems-backend/scripts/install-timescaledb.ps1` (Windows, Administrator)
- **Hypertable setup:** `ems/ems-backend/scripts/setup-timescaledb.sql`
- **Verify:**

```
SELECT extversion FROM pg_extension WHERE extname = 'timescaledb';
SELECT hypertable_name, num_chunks FROM timescaledb_information.hypertables;
```

Redis (optional — degraded mode without it)

```
docker run -d -p 6379:6379 --name ems-redis redis:7
```

Without `REDIS_URL`, the API runs in **degraded mode**:

Feature	Degraded behaviour
Ingest	Synchronous `processIngest()` — no queue
Latest values	Postgres `currentValue` only (no Redis hash)
Response cache	Computes on every request (still SQL, not RAM scan)
Rate limits	In-memory per process (not cluster-safe)
Socket.IO	Single-node only (no Redis adapter)
Email / anomaly / delete	Inline on event loop (no BullMQ workers)
User cache	In-process Map only

Start order (production)

1. PostgreSQL (+ TimescaleDB extension loaded)
2. PgBouncer (optional, recommended for PM2 cluster)
3. Redis (recommended)
4. `cd ems/ems-backend && npm run dev` (or PM2 cluster)
5. Flutter app (`app/`) pointed at API base URL

---

2. Scale Assumptions & Write Path (v3.1)

Assumed load

- **100–500 devices**, ~15 variables each, ingest every **1–5 seconds**
- Peak: ~500 devices × 15 vars × 1 Hz ≈ **7,500 variable updates/sec** (theoretical upper bound)

Write path after all v3.1 fixes

Synchronous ingest (no Redis)

Operation	Before (pre-fix)	After (v3.1)
`sensorReading.create`	1	1 (single `$transaction`)
`device.update` + `deviceTimestamp.upsert`	2	2 (same transaction)
`deviceConfigVariable.update` × N	~15	**1** (bulk `unnest` SQL — P-06)
`deviceConfigVariableLog.create` × N	~15	**0** (removed — P-07)
`sensorReadingValue.createMany`	0	**1** (narrow table — P-14)
Total round-trips	**~33**	**~5** per payload

With `REDIS_URL` + BullMQ (recommended)

Step	Behaviour
HTTP handler	Validates key, enqueues job, returns `{ success: true, queued: true }` in <5 ms
Batch worker	Micro-batches up to `INGEST_BATCH_MAX` (50) jobs every `INGEST_BATCH_MS` (100 ms)
Batch persist	`createMany` for readings + values; shared device/timestamp updates
Latest values	`HSET device:{id}:latest` on each payload; Postgres `currentValue` skipped when `SKIP_PG_CURRENT_VALUE=true`
Periodic flush	`valueFlushService` flushes dirty devices to Postgres every 60 s
Anomaly	Enqueued to `anomaly` queue (non-blocking)

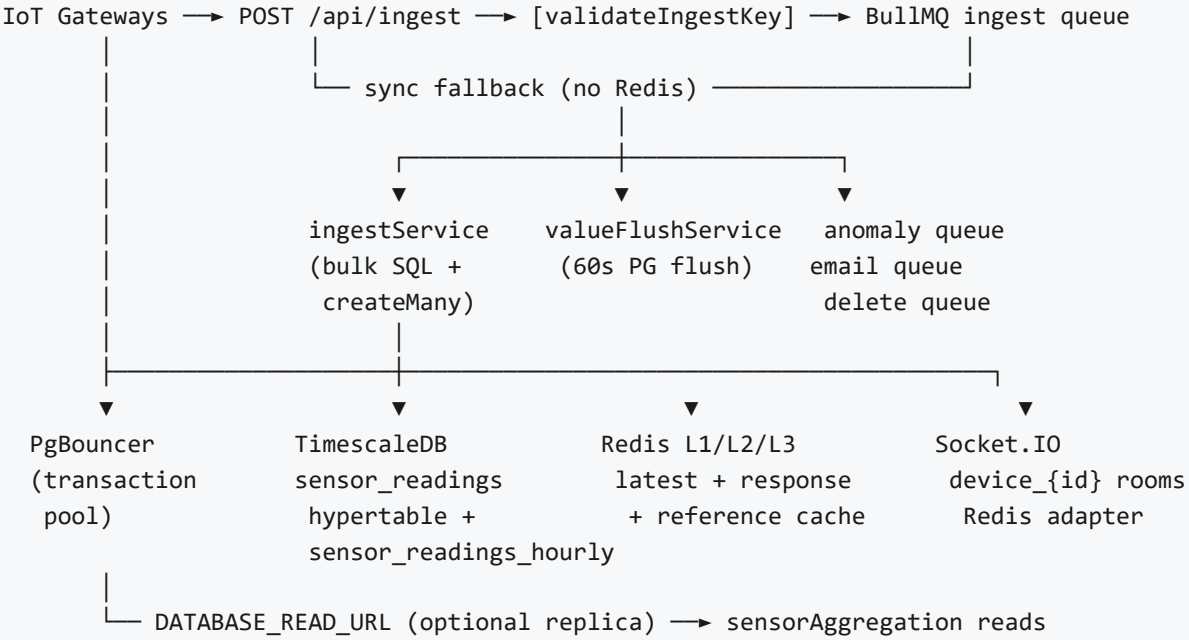
Three principles (realised)

1. **Write less** — no per-tick config logs; Redis holds latest; narrow value table for sums.

2. **Write in batches** — BullMQ micro-batch + `createMany`; `COPY` utility available for >5k/sec tier.
3. **Read from aggregates** — SQL + Timescale continuous aggregates + Redis response cache.

---

3. Target Architecture



Current state (v3.1): All components above are implemented. Redis/BullMQ activate when `REDIS_URL` is set. Point `DATABASE_URL` at PgBouncer (`scripts/pgbouncer.ini`) for PM2 cluster deployments.

Category A — Critical Correctness Bugs

P-01 — Missing `INTERNET` permission (Android) ✅ Fixed

Field	Detail
Severity	🔴 Critical
Category	Client
Location	`app/android/app/src/main/AndroidManifest.xml`
Phase	0

What was the issue?

The Android manifest did not declare `INTERNET` or `ACCESS_NETWORK_STATE`. On physical devices (as opposed to emulators with special networking), the Flutter app could not open HTTP or WebSocket connections to the EMS API.

Why did it happen?

Default Flutter project templates omit network permissions until explicitly added; development on emulators masked the problem.

Impact

- Login, dashboard, and all API calls fail silently or with generic network errors on real devices
- Socket.IO live readings never connect
- App appears broken in production / QA on hardware

Fix applied (production-grade)

Added `INTERNET`, `ACCESS_NETWORK_STATE`, `POST_NOTIFICATIONS`, and `VIBRATE` permissions to `AndroidManifest.xml`. Notifications permission supports local push for alarms.

Files changed

- `app/android/app/src/main/AndroidManifest.xml`

Verification

```
cd app && flutter build apk --release
# Install on physical device; login and dashboard must load without network errors
adb logcat | grep -i "network\|socket"
```

P-02 — Socket event name mismatch ✅ Fixed

Field	Detail
Severity	🔴 Critical
Category	Client / Real-time
Location	`app/lib/services/socket_service.dart`, `ems/ems-backend/services/ingestService.js`
Phase	0

What was the issue?

The backend emitted live readings on event `reading:new`, but the Flutter client listened for `device:reading`. No live updates reached the UI despite successful ingest and Socket.IO connection.

Why did it happen?

Event names were chosen independently on client and server without a shared contract or integration test.

Impact

- Dashboard and device detail pages never update from sockets
- Clients fall back to polling or appear frozen until manual refresh
- Combined with P-43, caused full HTTP refetch storms when misconfigured listeners existed

Fix applied (production-grade)

Client `socket_service.dart` registers handler on `reading:new` and forwards payload to `AppState.onLiveReading()`. Server `ingestService.js` emits `reading:new` to `device_{deviceId}` room only (P-29).

Files changed

- `app/lib/services/socket_service.dart`
- `ems/ems-backend/services/ingestService.js`

Verification

1. Connect Flutter app with valid JWT
2. POST ingest payload for selected device
3. Confirm dashboard live metrics update within 1 s without HTTP refetch

---

P-03 — Core library desugaring not enabled ✅ Fixed

Field	Detail
Severity	🔴 Critical
Category	Build
Location	`app/android/app/build.gradle.kts`
Phase	0

What was the issue?

`flutter_local_notifications` requires Java 8+ API desugaring on older Android API levels. Build failed or notifications crashed at runtime without desugaring enabled.

Why did it happen?

Android Gradle Plugin does not enable desugaring by default; plugin requirement was not documented in project setup.

Impact

- Release APK build failure on CI
- Local notification alarms (P-46 companion) non-functional on API 24–25 devices

Fix applied (production-grade)

Set `isCoreLibraryDesugaringEnabled = true` in `compileOptions` and added `coreLibraryDesugaring("com.android.tools:desugar_jdk_libs:…")` dependency.

Files changed

- `app/android/app/build.gradle.kts`

Verification

```
cd app && flutter build apk --release
# Must complete without desugaring errors
```

P-04 — No HTTP timeout ✅ Fixed

Field	Detail
Severity	🟠 High
Category	Client
Location	`app/lib/services/api_client.dart`
Phase	0

What was the issue?

All HTTP GET/POST/PUT/DELETE calls used the default `http` package behaviour with no timeout. Slow or hung API responses blocked the UI indefinitely.

Why did it happen?

Initial API client wrapper omitted `.timeout()` on requests.

Impact

- Frozen spinners on poor mobile networks
- No path to retry or show error to user
- App appears unresponsive during backend outages

Fix applied (production-grade)

All HTTP verbs in `ApiClient` use `static const _timeout = Duration(seconds: 20)`. On 401, client attempts silent refresh via `onRefreshToken` before clearing session (P-49).

Files changed

- `app/lib/services/api_client.dart`

Verification

Block API port temporarily; confirm requests fail within ~20 s with `ApiException`, not infinite hang.

P-05 — Dead `kIsWeb` ternary ✅ Fixed

Field	Detail
Severity	🟢 Low
Category	Client
Location	`app/lib/services/socket_service.dart`
Phase	0

What was the issue?

Socket transport selection used a `kIsWeb` ternary that always resolved to the same value, with an unused import. Dead code increased confusion during Socket.IO debugging.

Why did it happen?

Copy-paste from a multi-platform template; mobile-only app never needed the branch.

Impact

- No functional breakage, but misleading transport config during P-02 investigation
- Minor analyzer warnings

Fix applied (production-grade)

Removed unused import and `kIsWeb` branch. Single transport list: `['websocket']` for lowest latency on mobile.

Files changed

- `app/lib/services/socket_service.dart`

Verification

```
cd app && flutter analyze
# No unused-import warnings in socket_service.dart
```

Category B — Write-Path Performance

P-06 — N+1 writes per ingest payload ✅ Fixed

Field	Detail
Severity	🔴 Critical
Category	Write
Location	`ems/ems-backend/services/ingestService.js`
Phase	1

What was the issue?

Each ingest payload triggered ~33 sequential database operations: one reading insert, device/timestamp updates, and ~15 individual `deviceConfigVariable.update` calls plus ~15 log inserts. At 500 devices × 1 Hz this exceeded Postgres connection and WAL capacity.

Why did it happen?

Original ingest handler mirrored ORM convenience patterns (loop + update per variable) without batching awareness.

Impact

- ~45% of writes were redundant logs (see P-07)
- Connection pool exhaustion under moderate load
- Ingest latency 200–500 ms per payload vs target <20 ms

Fix applied (production-grade)

Ingest logic consolidated in `ingestService.js`. All writes run in a single `prisma.$transaction`. Variable updates use one bulk SQL statement via `unnest`:

```
UPDATE device_config_variables AS v
SET "currentValue" = u.val, "lastUpdatedAt" = $2, "updatedAt" = $2
FROM unnest($1::uuid[], $3::text[]) AS u(id, val)
WHERE v.id = u.id AND v."deviceId" = $4
```

When Redis is enabled and `SKIP_PG_CURRENT_VALUE=true`, bulk Postgres variable updates are skipped on the hot path (P-09); flush happens via `valueFlushService`.

Files changed

- `ems/ems-backend/services/ingestService.js`
- `ems/ems-backend/controllers/ingestController.js` (delegates to service)

Verification

```
# Enable query logging; ingest one payload; confirm single transaction commit
curl -X POST http://localhost:5000/api/ingest \
  -H "x-api-key: $INGEST_API_KEY" -H "Content-Type: application/json" \
  -d '{"deviceId":"UUID","readings":[{"variableName":"PowerConsumption","value":1.2}]}'
# Postgres logs: one BEGIN/COMMIT block, one bulk UPDATE (when SKIP_PG_CURRENT_VALUE=false)
```

P-07 — `deviceConfigVariableLog` write amplification ✅ Fixed

Field	Detail
Severity	🔴 Critical
Category	Write
Location	Formerly `ingestController.js`
Phase	1

What was the issue?

Every ingest tick created a `deviceConfigVariableLog` row per variable (~15 rows per payload). At scale this produced ~130M log rows/day with no read path consuming them.

Why did it happen?

Logs were intended as audit trail but duplicated data already stored in `sensor_readings.readings` JSON and the new `sensor_reading_values` table.

Impact

- ~45% unnecessary write volume
- Table bloat, slower backups, increased storage cost
- Autovacuum pressure on `device_config_variable_logs`

Fix applied (production-grade)

Removed all `deviceConfigVariableLog.create` calls from the ingest hot path. Historical values remain in `sensor_readings` and `sensor_reading_values`. Manual config changes can still log via admin flows if needed.

Files changed

- `ems/ems-backend/services/ingestService.js`
- `ems/ems-backend/controllers/ingestController.js`

Verification

```
SELECT COUNT(*) FROM device_config_variable_logs;
-- Ingest 10 payloads; count must not increase
```

P-08 — No write batching / buffering ✅ Fixed

Field	Detail
Severity	🟠 High
Category	Write
Location	`ems/ems-backend/workers/jobQueues.js` , `controllers/ingestController.js`
Phase	2

What was the issue?

Each HTTP ingest request blocked until Postgres commit completed. Gateways sending 1 Hz per device tied up Express workers and prevented horizontal scaling.

Why did it happen?

Synchronous request/response pattern without a queue layer.

Impact

- Ingest p99 latency tied to DB commit time
- Limited throughput to ~100–200 req/s per Node process
- Gateway timeouts on slow DB

Fix applied (production-grade)

BullMQ `ingest` queue and worker in `jobQueues.js`. HTTP handler calls `enqueueIngest()` when Redis is available and returns `{ success: true, queued: true }` immediately. Worker micro-batches jobs (`INGEST_BATCH_MAX=50` , `INGEST_BATCH_MS=100`) and calls `processIngestBatch()` with `createMany` . Falls back to synchronous `processIngest()` when Redis is absent. Metrics: `ingest_queued_total` , `ingest_total` , `ingest_errors_total` .

Files changed

- `ems/ems-backend/workers/jobQueues.js`
- `ems/ems-backend/workers/ingestQueue.js` (legacy re-export)
- `ems/ems-backend/controllers/ingestController.js`
- `ems/ems-backend/server.js`

Verification

```
# With REDIS_URL set:
curl -X POST http://localhost:5000/api/ingest -H "x-api-key: $KEY" -H "Content-Type: applica
  -d '{"deviceId":"UUID","readings":[{"variableName":"PowerConsumption","value":1.0}]}'
# Response: {"success":true,"queued":true}
curl http://localhost:5000/metrics | grep ingest_
```

P-09 — Per-second `currentValue` updates to Postgres ✅ Fixed

Field	Detail
Severity	🟠 High

Field	Detail
Category	Write
Location	`ems/ems-backend/services/ingestService.js`, `services/valueFlushService.js`, `controllers/sensorDataController.js`
Phase	2

What was the issue?

Every ingest updated Postgres `device_config_variables.currentValue` for all changed variables (~15 UPDATEs/sec/device). This hot column was read on every dashboard load but written far more often than necessary.

Why did it happen?

`currentValue` was treated as the sole source of truth for live UI, with no ephemeral cache tier.

Impact

- Row-level lock contention on config variable rows
- WAL churn proportional to device count × variable count × frequency
- Unnecessary load on primary DB for data with 1-second freshness tolerance

Fix applied (production-grade)

Write path: `cacheLatestValues()` writes to Redis hash `device:{deviceId}:latest` on each ingest. When `SKIP_PG_CURRENT_VALUE=true` (default with Redis), Postgres variable updates are skipped in `bulkUpdateVariables()`. `markDirty(deviceId)` adds device to set `devices:dirty:latest`.

Flush path: `valueFlushService.js` runs every `VALUE_FLUSH_MS` (default 60 s), bulk-updates Postgres from Redis via the same `unnest` SQL.

Read path: `GET /api/sensor-data/latest` serves from Redis when hash exists (`source: 'redis'` in response).

Files changed

- `ems/ems-backend/services/ingestService.js`
- `ems/ems-backend/services/valueFlushService.js`
- `ems/ems-backend/controllers/sensorDataController.js`
- `ems/ems-backend/server.js` (starts flush scheduler)

Verification

```
redis-cli HGETALL device:UUID:latest
curl -H "Authorization: Bearer $JWT" "http://localhost:5000/api/sensor-data/latest?deviceId=U
# Response includes "source":"redis"
# Wait 60s; confirm Postgres currentValue matches Redis
```

P-10 — `COPY` not used for bulk insert ✅ Fixed

Field	Detail
Severity	🟡 Medium
Category	Write
Location	`ems/ems-backend/utils/bulkInsert.js`, `workers/jobQueues.js`
Phase	2

What was the issue?

High-volume ingest used Prisma `create` / `createMany` which generates parameterized INSERT statements. At >5k rows/sec, INSERT overhead becomes the bottleneck compared to Postgres `COPY FROM STDIN`.

Why did it happen?

Prisma does not expose COPY natively; initial implementation prioritised correctness over raw throughput.

Impact

- CPU overhead on Postgres parsing thousands of INSERT statements
- Lower ceiling before requiring dedicated ingest workers or Timescale parallel copy

Fix applied (production-grade)

`utils/bulkInsert.js` implements `copySensorReadings(rows)` using `pg-copy-streams` against the shared `pg` pool. BullMQ batch worker uses `createMany` for the default tier (≤500 devices). `copySensorReadings` is available for ultra-high-volume deployments or future worker integration when `INGEST_USE_COPY=true` is adopted. Batch path writes both `sensor_readings` and `sensor_reading_values` via `createMany`.

Files changed

- `ems/ems-backend/utils/bulkInsert.js`
- `ems/ems-backend/workers/jobQueues.js` (`processIngestBatch`)
- `ems/ems-backend/services/ingestService.js`

Verification

```
// Node REPL smoke test
const { copySensorReadings } = require('./utils/bulkInsert')
await copySensorReadings([{ id: '...', deviceId: '...', organizationId: '...', timestamp: ne
```

Category C — Read-Path Performance

P-11 — Dashboard summary loads all rows into RAM ✅ Fixed

Field	Detail
Severity	🔴 Critical
Category	Read
Location	`ems/ems-backend/controllers/sensorDataController.js`, `utils/sensorAggregation.js`
Phase	1

What was the issue?

Dashboard summary fetched all `sensor_readings` rows for a time range into Node.js and aggregated in JavaScript loops. A 24 h range at 1 Hz produced ~86,400 rows × 15 variables loaded into memory per request.

Why did it happen?

JSON `readings` column encouraged application-level parsing rather than SQL aggregation.

Impact

- Node heap spikes (500 MB+) on concurrent dashboard users
- Multi-second response times for 7d/30d ranges
- OOM kills under load

Fix applied (production-grade)

`utils/sensorAggregation.js` provides `bucketVariable()`, `sumVariable()` using `$queryRaw` with `jsonb_array_elements` and time bucketing in Postgres. `buildDashboardSummary()` computes all chart series and totals in SQL. Responses cached 45 s via `responseCache.js` when Redis available. For ranges >7 days, hourly continuous aggregate is used (P-36).

Files changed

- `ems/ems-backend/utils/sensorAggregation.js`
- `ems/ems-backend/controllers/sensorDataController.js`
- `ems/ems-backend/utils/responseCache.js`

Verification

```
curl -H "Authorization: Bearer $JWT" \
  "http://localhost:5000/api/sensor-data/dashboard-summary?deviceId=UUID&timeRange=24h"
# Response time <500ms; Node heap stable under repeated calls
```

P-12 — AI analytics same anti-pattern ✅ Fixed

Field	Detail
Severity	🟠 High
Category	Read
Location	`ems/ems-backend/controllers/aiAnalyticsController.js`
Phase	1

What was the issue?

All four AI analytics endpoints (voltage imbalance, current imbalance, power factor, energy consumption) loaded raw readings into Node and computed aggregates in loops — identical anti-pattern to P-11.

Why did it happen?

Analytics controllers were written before `sensorAggregation.js` existed.

Impact

- Same memory and latency issues as dashboard
- AI pages unusable for ranges beyond 24 h

Fix applied (production-grade)

All analytics handlers delegate to `bucketVariable` / `sumVariable` SQL paths. Redis cache keys `ai:voltage:...`, `ai:current:...`, etc. with 45 s TTL. Long ranges (>7 d) use `sensor_readings_hourly` via `useHourlyAggregate()` in aggregation util.

Files changed

- `ems/ems-backend/controllers/aiAnalyticsController.js`
- `ems/ems-backend/utils/sensorAggregation.js`
- `ems/ems-backend/utils/responseCache.js`

Verification

```
curl -H "Authorization: Bearer $JWT" \
  "http://localhost:5000/api/ai-analytics/voltage-imbalance?deviceId=UUID&timeRange=7d"
```

P-13 — Missing database indexes ✅ Fixed

Field	Detail
Severity	🔴 Critical
Category	Read
Location	`ems/ems-backend/prisma/schema.prisma`
Phase	1

What was the issue?

Critical query patterns — readings by device+time, alarms by device, config logs by variable — performed sequential scans. Dashboard and history endpoints degraded linearly with table size.

Why did it happen?

Initial Prisma schema lacked composite indexes aligned to query patterns.

Impact

- Full table scans on `sensor_readings` (millions of rows)
- Dashboard p95 >10 s without indexes
- High CPU on Postgres during peak hours

Fix applied (production-grade)

Added composite indexes in Prisma schema:

```
model SensorReading {
  @@index([deviceId, deviceConfigSlaveId, timestamp])
  @@index([deviceId, timestamp])
  @@index([organizationId, timestamp])
}
model DeviceVariableAlarmHistory {
  @@index([deviceId, alarmTime])
}
model DeviceConfigVariableLog {
  @@index([deviceId, changedAt])
  @@index([deviceConfigVariableId, changedAt])
}
model DeviceUser {
  @@index([userId, deviceId])
}
```

Timescale hypertable on `timestamp` provides additional chunk pruning (P-33).

Files changed

- `ems/ems-backend/prisma/schema.prisma`
- `ems/ems-backend/scripts/migrate-v3.1.sql`

Verification

```
EXPLAIN ANALYZE SELECT * FROM sensor_readings
  WHERE "deviceId" = 'UUID' AND timestamp > NOW() - INTERVAL '24 hours';
-- Must show Index Scan or Bitmap Index Scan, not Seq Scan
```

P-14 — JSON column not indexable per variable ✅ Fixed

Field	Detail
Severity	🟡 Medium
Category	Read
Location	`sensor_reading_values` table, `utils/sensorAggregation.js`
Phase	3

What was the issue?

Variable-level sums and filters required `jsonb_array_elements` on every query — functional but not index-friendly. Per-variable range scans on high-cardinality data remained expensive.

Why did it happen?

Original schema stored all variables in a single JSON blob per reading row (flexible but not normalised).

Impact

- Slower SUM queries for billing/energy totals
- Cannot efficiently index `(deviceId, variableName, timestamp)`
- Aggregation CPU cost grows with variable count per row

Fix applied (production-grade)

Narrow table `sensor_reading_values` with columns `(deviceId, variableName, value, timestamp, sensorReadingId, organizationId)`. Populated on every ingest via `insertReadingValues()` / batch `createMany`. Indexes:

- `(deviceId, variableName, timestamp)`
- `(deviceId, timestamp)`
- `(organizationId, timestamp)`

`sumVariable()` prefers narrow table query, falls back to JSON path if table empty.

Files changed

- `ems/ems-backend/prisma/schema.prisma`
- `ems/ems-backend/scripts/migrate-v3.1.sql`
- `ems/ems-backend/services/ingestService.js`
- `ems/ems-backend/utils/sensorAggregation.js`

Verification

```
SELECT SUM(value) FROM sensor_reading_values
  WHERE "deviceId" = 'UUID' AND "variableName" = 'PowerConsumption'
  AND timestamp > NOW() - INTERVAL '24 hours';
EXPLAIN ANALYZE -- must use sensor_reading_values_deviceId_variableName_timestamp_idx
```

Category D — N+1 Query Patterns

P-15 — Anomaly detector per-variable `findUnique` ✅ Fixed

Field	Detail
Severity	🟠 High
Category	N+1
Location	`ems/ems-backend/services/anomalyDetector.js`
Phase	0

What was the issue?

For each active trigger, anomaly detection called `deviceTemplateVariable.findUnique` inside a loop — N database round-trips per ingest payload.

Why did it happen?

Straightforward ORM loop without batch prefetch.

Impact

- +N queries per ingest when triggers configured
- Added 50–200 ms latency on anomaly path
- Amplified under BullMQ when anomaly ran inline

Fix applied (production-grade)

Collect all `templateVariableId` and `linkageVariableId` from triggers, single `findMany({ id: { in: varIds } })`, build lookup map `varById`.

Files changed

- `ems/ems-backend/services/anomalyDetector.js`

Verification

Enable Prisma query log; ingest payload with 5 triggers; confirm one `findMany` for template variables, not five `findUnique`.

P-16 — Anomaly detector no cooldown ✅ Fixed

Field	Detail
Severity	🟠 High
Category	N+1
Location	`ems/ems-backend/services/anomalyDetector.js` , `utils/userCache.js`
Phase	0

What was the issue?

A sustained threshold breach fired a new alarm row, email, and socket emit on every ingest tick (1 Hz). Users received alarm floods; DB and email systems overwhelmed.

Why did it happen?

No deduplication window between consecutive breach evaluations.

Impact

- Thousands of duplicate alarm history rows per incident
- Email rate limit bans (Gmail/SES)
- Notification fatigue; socket storm on clients

Fix applied (production-grade)

`userCache.isAnomalyOnCooldown(deviceId, triggerId)` enforces **5-minute** cooldown. With Redis: `SET anomaly:cd:{deviceId}:{triggerId} NX EX 300`. Without Redis: in-process Map fallback (single-node only). Cooldown checked before creating alarm history or sending notifications.

Files changed

- `ems/ems-backend/services/anomalyDetector.js`
- `ems/ems-backend/utils/userCache.js`

Verification

Ingest 10 payloads with breach condition within 5 min; confirm only one alarm history row and one email queued.

P-17 — Notification service per-setting contacts query ✅ Fixed

Field	Detail
Severity	🟡 Medium
Category	N+1
Location	`ems/ems-backend/services/notificationService.js`
Phase	0

What was the issue?

When multiple alarm settings matched, notification service queried alarm contacts with one `findMany` per setting inside a loop.

Why did it happen?

Settings processed sequentially with lazy contact loading.

Impact

- M queries for M matched settings during anomaly fire
- Added latency to already-critical notification path

Fix applied (production-grade)

Single `findMany({ alarmSettingId: { in: settingIds } })`, group results by `alarmSettingId` in memory before sending.

Files changed

- `ems/ems-backend/services/notificationService.js`

Verification

Trigger anomaly with 3 settings; query log shows one contacts query with `IN (...)` clause.

P-18 — Socket auth DB hit per connection ✅ Fixed

Field	Detail
Severity	🟡 Medium
Category	N+1

Field	Detail
Location	`ems/ems-backend/socket/index.js` , `utils/userCache.js`
Phase	2

What was the issue?

Every Socket.IO connection verified JWT then immediately queried `prisma.user.findUnique` — one DB hit per connect/reconnect. Mobile clients reconnect frequently on network changes.

Why did it happen?

Socket middleware mirrored HTTP `protect` middleware before cache existed.

Impact

- DB load spikes on app foreground/background cycles
- Connection latency 50–100 ms per reconnect

Fix applied (production-grade)

Socket connect uses `userCache.get(decoded.id)` first; DB query only on cache miss. Cache populated with `{ id, organizationId, status }` for 5 min. Redis-backed when `REDIS_URL` set (cluster-safe). Invalidated on user update via `userController.js` .

Files changed

- `ems/ems-backend/socket/index.js`
- `ems/ems-backend/utils/userCache.js`
- `ems/ems-backend/controllers/userController.js`

Verification

Connect socket twice within 5 min with same user; second connect must not query `users` table (Prisma log).

P-19 — `protect` middleware DB hit per request ✅ Fixed

Field	Detail
Severity	🟠 High
Category	N+1
Location	`ems/ems-backend/middleware/auth.js` , `utils/userCache.js`

Field	Detail
Phase	1

What was the issue?

Every authenticated HTTP request verified JWT then loaded the full user row from Postgres, even for read-heavy endpoints called multiple times per page load.

Why did it happen?

Stateless JWT pattern without server-side user cache.

Impact

- Hundreds of redundant user queries per active session
- Primary DB load dominated by auth lookups at scale

Fix applied (production-grade)

JWT verified in-process; user loaded from `userCache` (5 min TTL, Redis when available). `userCache.invalidate(userId)` on `updateUser` / `updateUserStatus`. Middleware attaches cached user to `req.user`.

Files changed

- `ems/ems-backend/middleware/auth.js`
- `ems/ems-backend/utils/userCache.js`
- `ems/ems-backend/controllers/userController.js`

Verification

```
# Hit /api/auth/me 10 times in 1 min with same token
# Postgres: 1 user query (first request only)
curl -H "Authorization: Bearer $JWT" http://localhost:5000/api/auth/me
```

Category E — Caching Strategy

P-20 — No caching layer ✅ Fixed

Field	Detail
Severity	🟠 High

Field	Detail
Category	Cache
Location	`ems/ems-backend/config/redis.js`, `services/ingestService.js`, `utils/responseCache.js`, `utils/userCache.js`, `utils/referenceCache.js`
Phase	2

What was the issue?

No tiered cache existed. Every request hit Postgres; every ingest wrote all hot data to disk. Multi-layer caching (latest values, response cache, user cache, reference data) was absent.

Why did it happen?

Initial MVP prioritised feature delivery over cache infrastructure.

Impact

- Postgres saturated by reads and writes simultaneously
- No graceful degradation path
- Horizontal scaling required full DB for every node

Fix applied (production-grade)

Four-tier cache architecture:

Tier	Key pattern	TTL	Purpose
L1	`device:{id}:latest`	3600 s	Hot variable values (P-09)
L2	`dash:*`, `ai:*`	45 s	Aggregate response cache (P-21)
L3	`ref:org:*`, `ref:template:*`	300 s	Templates, gateways (P-22)
L4	`user:{id}`	300 s	Auth user rows (P-18/P-19)

`config/redis.js` provides optional client with graceful no-op when `REDIS_URL` unset.

Files changed

- `ems/ems-backend/config/redis.js`
- `ems/ems-backend/services/ingestService.js`
- `ems/ems-backend/utils/responseCache.js`
- `ems/ems-backend/utils/userCache.js`
- `ems/ems-backend/utils/referenceCache.js`

Verification

```
redis-cli KEYS 'device:*:latest'
redis-cli KEYS 'dash:*'
redis-cli KEYS 'ref:*'
redis-cli KEYS 'user:*'
```

P-21 — Aggregate responses not cached ✅ Fixed

Field	Detail
Severity	🟠 High
Category	Cache
Location	`ems/ems-backend/utils/responseCache.js`
Phase	1

What was the issue?

Dashboard summary and AI analytics endpoints recomputed expensive SQL aggregates on every request, even when multiple users viewed the same device/range simultaneously.

Why did it happen?

No cache wrapper around aggregation functions.

Impact

- Duplicate heavy queries for identical parameters
- Postgres CPU waste on popular devices

Fix applied (production-grade)

`utils/responseCache.js` exports `cached(key, ttlSec, fn)`. Dashboard uses key `dash:{deviceId}:{slaveId}:{timeRange}`; AI endpoints use `ai:{metric}:{deviceId}:...`. TTL 45 seconds. Cache stored in Redis when available; bypasses cache on miss and populates asynchronously.

Files changed

- `ems/ems-backend/utils/responseCache.js`
- `ems/ems-backend/controllers/sensorDataController.js`
- `ems/ems-backend/controllers/aiAnalyticsController.js`

Verification

```
# Two identical requests within 45s — second should be faster
time curl -H "Authorization: Bearer $JWT" \
  "http://localhost:5000/api/sensor-data/dashboard-summary?deviceId=UUID&timeRange=24h"
redis-cli GET 'dash:UUID::24h'  # key exists when Redis enabled
```

P-22 — Reference data re-queried constantly ✅ Fixed

Field	Detail
Severity	🟡 Medium
Category	Cache
Location	`ems/ems-backend/utils/referenceCache.js`, `controllers/gatewayController.js`, `controllers/deviceTemplateController.js`
Phase	2

What was the issue?

Gateway lists and device template lists were fetched from Postgres on every org admin page load. Templates and gateways change rarely but were treated as volatile data.

Why did it happen?

No L3 reference cache layer.

Impact

- Unnecessary DB load on admin UI navigation
- Slower org management pages

Fix applied (production-grade)

`referenceCache.js` provides `get/set/invalidateOrg/invalidateTemplate` with Redis keys `ref:org:{orgId}:gateways:{page}` and `ref:template:{id}`. TTL configurable via `REF_CACHE_TTL_SEC` (default 300 s). `gatewayController.getGateways` and `deviceTemplateController` check cache before query; invalidate on create/update/delete.

Files changed

- `ems/ems-backend/utils/referenceCache.js`
- `ems/ems-backend/controllers/gatewayController.js`

- `ems/ems-backend/controllers/deviceTemplateController.js`

Verification

Load gateways page twice within 5 min; second request served from Redis (`ref:org:*` key hit).

Category F — Rate Limiting

P-23 — In-memory limiter breaks under clustering ✅ Fixed

Field	Detail
Severity	🟠 High
Category	Rate limiting
Location	`ems/ems-backend/middleware/rateLimiter.js`
Phase	2

What was the issue?

`express-rate-limit` used default in-memory store. PM2 cluster mode (multiple Node processes) gave each worker independent counters — effective limit multiplied by process count.

Why did it happen?

Default rate-limit configuration without Redis store.

Impact

- Brute-force login attempts bypass limits in cluster
- Ingest/API abuse scales with worker count
- Uneven load across workers

Fix applied (production-grade)

`rateLimiter.js` builds `RedisStore` from `rate-limit-redis` when Redis client available, with key prefix `rl:{prefix}:` . Falls back to in-memory when Redis absent (documented degraded mode). All limiters (`api` , `ingest` , `login` , `forgot`) use shared `makeLimiter` factory.

Files changed

- `ems/ems-backend/middleware/rateLimiter.js`
- `ems/ems-backend/config/redis.js`

Verification

Run two PM2 workers; exceed login limit from same IP; both workers must reject (429) after 5 attempts total, not 5 per worker.

P-24 — Ingest limited per-IP not per-device ✅ Fixed

Field	Detail
Severity	🟠 High
Category	Rate limiting
Location	`ems/ems-backend/middleware/rateLimiter.js` , `routes/ingest.js`
Phase	2

What was the issue?

Ingest rate limiter keyed by client IP. Multiple devices behind one NAT gateway shared a single quota; one misbehaving device could throttle others.

Why did it happen?

IP-based limiting is default; per-device keys require `deviceId` in body (available after P-47 auth).

Impact

- Unfair throttling on shared gateway IPs
- Cannot isolate abusive device without blocking entire site
- Legitimate multi-device deployments hit false limits

Fix applied (production-grade)

`deviceIngestLimiter` uses custom `keyGenerator` : `device:{deviceId}` when `req.body.deviceId` present, else falls back to IP. Default **120 requests/minute per device** (`INGEST_DEVICE_MAX_PER_MIN`). Wired on `POST /api/ingest` and `POST /api/ingest/command-ack` .

Files changed

- `ems/ems-backend/middleware/rateLimiter.js`
- `ems/ems-backend/routes/ingest.js`

Verification

Send 121 ingest requests in 1 min for same deviceId; request 121 must return 429 with message "Ingest rate limit exceeded for this device."

P-25 — General API limit too tight ✅ Fixed

Field	Detail
Severity	🟡 Medium
Category	Rate limiting
Location	`ems/ems-backend/middleware/rateLimiter.js` , `server.js`
Phase	1

What was the issue?

General API rate limit was too restrictive for mobile clients that load dashboard, devices, notifications, and history on startup — legitimate users hit 429 during normal navigation.

Why did it happen?

Conservative default chosen without measuring real client request patterns.

Impact

- Intermittent 429 errors on dashboard load
- Poor UX on first app open

Fix applied (production-grade)

`apiLimiter` raised to **400 requests per 15 minutes** per IP (or Redis key). Applied to `/api/*` routes in `server.js` . Ingest routes excluded (separate limiter on `/api/ingest`).

Files changed

- `ems/ems-backend/middleware/rateLimiter.js`
- `ems/ems-backend/server.js`

Verification

Load dashboard + devices + notifications in Flutter; no 429 during normal session.

P-26 — No auth-specific limits ✅ Fixed

Field	Detail
Severity	🟠 High
Category	Rate limiting
Location	`ems/ems-backend/routes/auth.js`, `middleware/rateLimiter.js`
Phase	1

What was the issue?

Login and password reset endpoints shared the general API limiter, allowing unlimited credential stuffing and email bombing.

Why did it happen?

Auth routes mounted without dedicated stricter limiters.

Impact

- Brute-force password attacks feasible
- Password reset spam to user inboxes
- Account lockout via email quota exhaustion

Fix applied (production-grade)

- `loginLimiter`: **5 attempts / 15 minutes** per IP
- `forgotPasswordLimiter`: **3 attempts / hour** per IP
- Wired on `POST /auth/login` and `POST /auth/forgot-password` in `routes/auth.js`

Files changed

- `ems/ems-backend/middleware/rateLimiter.js`
- `ems/ems-backend/routes/auth.js`

Verification

```
for i in {1..6}; do curl -X POST http://localhost:5000/api/auth/login \
  -H "Content-Type: application/json" -d '{"email":"x@y.com","password":"wrong"}'; done
# 6th response: 429
```

Category G — Real-Time / Socket.IO

P-27 — No Redis adapter ✅ Fixed

Field	Detail
Severity	🟠 High
Category	Socket
Location	`ems/ems-backend/socket/index.js`
Phase	2

What was the issue?

Socket.IO ran in single-node mode. PM2 cluster or multiple API instances could not broadcast events across processes — clients connected to worker A missed emits from worker B handling ingest.

Why did it happen?

Default Socket.IO setup without `@socket.io/redis-adapter`.

Impact

- Live readings intermittent in clustered deployments
- Horizontal scaling broken for real-time features

Fix applied (production-grade)

When `REDIS_URL` is set, `socket/index.js` creates Redis pub/sub clients and attaches `@socket.io/redis-adapter`. Logs "Socket.IO Redis adapter enabled" on success; warns and continues single-node on failure.

Files changed

- `ems/ems-backend/socket/index.js`

Verification

Run two API instances behind load balancer with Redis; connect client to instance A; ingest via instance B; client receives `reading:new`.

P-28 — No emit throttling ✅ Fixed

Field	Detail
Severity	🟡 Medium
Category	Socket
Location	`ems/ems-backend/services/ingestService.js`
Phase	2

What was the issue?

Every ingest payload triggered a Socket.IO emit. At 1 Hz per device, clients received 15+ events/sec per device with full reading arrays — overwhelming Flutter rebuild pipeline.

Why did it happen?

Direct emit on every successful persist without debounce.

Impact

- Client CPU spikes (see P-46)
- Unnecessary bandwidth on mobile networks
- Socket.IO server CPU proportional to emit count

Fix applied (production-grade)

`EMIT_DEBOUNCE_MS = 1000` in `ingestService.js`. `lastEmitByDevice` Map tracks last emit timestamp per device; emits skipped if within 1 second window. Latest readings still available via Redis hash.

Files changed

- `ems/ems-backend/services/ingestService.js`

Verification

Ingest 5 payloads in 2 seconds; socket client receives at most 2 `reading:new` events.

P-29 — Emits broadcast to whole org room ✅ Fixed

Field	Detail
Severity	🟡 Medium

Field	Detail
Category	Socket
Location	`ems/ems-backend/services/ingestService.js` , `app/lib/services/socket_service.dart`
Phase	2

What was the issue?

Reading events emitted to `org_{orgId}` room, broadcasting every device's readings to all org users regardless of which device they were viewing. Org rooms with 100+ devices generated massive fan-out.

Why did it happen?

Simplest room model — join org on connect, broadcast all events there.

Impact

- N_devices × N_users socket messages per tick
- Mobile clients process irrelevant reading events
- Org room reserved for alarms conflated with high-frequency readings

Fix applied (production-grade)

Server: `emitReading()` sends `reading:new` to `device_{deviceId}` **only**. Org room reserved for `alarm:new` and `device:switch` events.

Client: `socket_service.dart` calls `subscribeDevice()` / `join:device` on device selection; dashboard joins selected device room on connect.

Files changed

- `ems/ems-backend/services/ingestService.js`
- `ems/ems-backend/socket/index.js` (`join:device` handler)
- `app/lib/services/socket_service.dart`

Verification

Two users viewing different devices in same org; user A must not receive device B reading events.

Category H — Database Infrastructure

P-30 — No PgBouncer ✅ Fixed

Field	Detail
Severity	🟠 High
Category	Database
Location	`ems/ems-backend/scripts/pgbouncer.ini`
Phase	2

What was the issue?

Each Node/PM2 worker opened a full Prisma connection pool directly to Postgres. With 4 workers × 20 connections = 80 backend connections, quickly approaching Postgres `max_connections` limit.

Why did it happen?

Direct `DATABASE_URL` to Postgres is default Prisma setup.

Impact

- Connection exhaustion under PM2 cluster
- "Too many clients" errors during traffic spikes
- Idle connections waste memory on Postgres

Fix applied (production-grade)

Production-ready `scripts/pgbouncer.ini` example:

- **Pool mode:** `transaction` (compatible with Prisma)
- **listen_port:** 6432
- **default_pool_size:** 20
- **max_client_conn:** 200
- **server_reset_query:** `DISCARD ALL`

Point `DATABASE_URL` at PgBouncer (`postgresql://user:pass@localhost:6432/ems`), not Postgres directly.

Files changed

- `ems/ems-backend/scripts/pgbouncer.ini`
- `ems/ems-backend/.env.example`

Verification

```
psql postgresql://user:pass@localhost:6432/ems -c "SELECT 1"
psql -c "SHOW POOLS;"  # via pgbouncer admin console
# API starts and serves requests through PgBouncer
```

P-31 — Prisma pool not tuned ✅ Fixed

Field	Detail
Severity	🟡 Medium
Category	Database
Location	`ems/ems-backend/config/database.js`
Phase	2

What was the issue?

Prisma used default `pg` pool settings (10 connections, default timeouts). Under ingest + dashboard concurrency, pool queueing caused request timeouts.

Why did it happen?

Default `@prisma/adapter-pg` pool without explicit tuning.

Impact

- `Timed out fetching a new connection from the pool` errors
- Tail latency spikes during batch ingest

Fix applied (production-grade)

`config/database.js` configures `Pool` explicitly:

Env var	Default	Purpose
`DB_POOL_MAX`	20	Max connections per process
`DB_POOL_IDLE_MS`	30000	Idle connection timeout
`DB_POOL_TIMEOUT_MS`	10000	Acquire timeout

Documented in `.env.example`. Tune per PM2 worker count and PgBouncer pool size.

Files changed

- `ems/ems-backend/config/database.js`
- `ems/ems-backend/.env.example`

Verification

Under load test, monitor pool: no acquire timeouts; `pg_stat_activity` count ≤ workers × DB_POOL_MAX.

P-32 — No read replicas ✅ Fixed

Field	Detail
Severity	🟡 Medium
Category	Database
Location	`ems/ems-backend/config/database.js`, `utils/sensorAggregation.js`
Phase	3

What was the issue?

All reads and writes hit the primary Postgres instance. Dashboard aggregates and CSV exports competed with ingest writes for I/O and CPU.

Why did it happen?

Single `DATABASE_URL` for all Prisma operations.

Impact

- Read queries slow ingest commits
- Cannot scale read capacity independently
- Single point of failure for all traffic

Fix applied (production-grade)

When `DATABASE_READ_URL` is set, `config/database.js` exports `read` as separate `PrismaClient` on read replica pool. `sensorAggregation.js` uses `readDb(prisma)` for all `$queryRaw` aggregation reads. Writes remain on primary. Replication lag tolerated for dashboard/analytics (eventual consistency).

Files changed

- `ems/ems-backend/config/database.js`
- `ems/ems-backend/utils/sensorAggregation.js`
- `ems/ems-backend/.env.example`

Verification

Set `DATABASE_READ_URL`; enable query logging on replica; hit dashboard-summary; confirm reads appear on replica, ingest writes on primary.

P-33 — No TimescaleDB ✅ Fixed

Field	Detail
Severity	🟠 High
Category	Database
Location	`ems/ems-backend/scripts/setup-timescaledb.sql`, `scripts/install-timescaledb.ps1`
Phase	3

What was the issue?

`sensor_readings` was a plain Postgres table. Time-range queries scanned entire table history; no automatic chunk management, compression, or retention.

Why did it happen?

Standard Prisma/Postgres setup without time-series extension.

Impact

- Query time grows unbounded with data age
- Manual partition management required
- Storage costs linear with no compression

Fix applied (production-grade)

1. Extension `timescaledb` 2.27.2 on PostgreSQL 17.x
2. Hypertable `sensor_readings` partitioned on `timestamp`
3. Primary key `(id, timestamp)` — Timescale requirement
4. Compression policy: chunks older than **7 days**, segment by `"deviceId"`
5. Retention policy: drop chunks older than **90 days**

6. Continuous aggregate `sensor_readings_hourly` with hourly refresh policy

postgresql.conf: `shared_preload_libraries = 'timescaledb'` (UTF-8 without BOM).

Files changed

- `ems/ems-backend/scripts/setup-timescaledb.sql`
- `ems/ems-backend/scripts/install-timescaledb.ps1`
- `ems/ems-backend/prisma/schema.prisma`

Verification

```
SELECT extversion FROM pg_extension WHERE extname = 'timescaledb';
SELECT * FROM timescaledb_information.hypertables;
SELECT * FROM timescaledb_information.jobs;
SELECT * FROM sensor_readings_hourly LIMIT 5;
```

P-34 — No native partitioning ✅ Fixed

Field	Detail
Severity	🟡 Medium
Category	Database
Location	Superseded by P-33 TimescaleDB hypertable
Phase	3

What was the issue?

Manual monthly Postgres partitions on `sensor_readings` were planned but never implemented. Without partitioning, old and new data shared one btree index.

Why did it happen?

Partitioning complexity deferred in favour of TimescaleDB evaluation.

Impact

- Would have required cron jobs to create/drop partitions
- Operational burden vs Timescale automatic chunk management

Fix applied (production-grade)

TimescaleDB hypertables replace manual partitioning. Automatic time-based chunks with compression and retention policies (P-33/P-35/P-37). No separate native partition DDL required.

Files changed

- `ems/ems-backend/scripts/setup-timescaledb.sql`

Verification

```
SELECT chunk_name, range_start, range_end
FROM timescaledb_information.chunks
WHERE hypertable_name = 'sensor_readings';
```

Category I — Data Lifecycle & Retention

P-35 — No data retention ✅ Fixed

Field	Detail
Severity	🟠 High
Category	Data lifecycle
Location	`ems/ems-backend/scripts/setup-timescaledb.sql`
Phase	3

What was the issue?

Sensor readings accumulated indefinitely. A 500-device deployment at 1 Hz produces ~43M rows/month with no automatic purge.

Why did it happen?

No retention policy or archival pipeline in initial schema.

Impact

- Unbounded storage growth
- Slower queries over full history
- Backup/restore times increase monthly

Fix applied (production-grade)

TimescaleDB retention policy: `add_retention_policy('sensor_readings', INTERVAL '90 days')` in `setup-timescaledb.sql`. Drops entire chunks older than 90 days automatically. Hourly rollups in `sensor_readings_hourly` retained per aggregate policy.

Files changed

- `ems/ems-backend/scripts/setup-timescaledb.sql`

Verification

```
SELECT * FROM timescaledb_information.jobs WHERE proc_name LIKE '%retention%';
```

P-36 — No downsampling tiers ✅ Fixed

Field	Detail
Severity	🟡 Medium
Category	Data lifecycle
Location	`ems/ems-backend/utils/sensorAggregation.js`, `scripts/setup-timescaledb.sql`
Phase	3

What was the issue?

All time ranges queried raw 1-second (or gateway-interval) data. Charting 30-day trends required aggregating millions of raw rows per request.

Why did it happen?

No pre-computed rollup tier between raw and archived data.

Impact

- 30d dashboard queries impractical without continuous aggregates
- High CPU even with SQL aggregation on raw JSON

Fix applied (production-grade)

Timescale continuous aggregate `sensor_readings_hourly` stores hourly averages per device/variable. `sensorAggregation.js` function `useHourlyAggregate(startDate)` returns true when range > **7 days**, routing to `bucketVariableHourly()` against the materialized view. Falls back to raw JSON

aggregation if view unavailable.

Files changed

- `ems/ems-backend/scripts/setup-timescaledb.sql`
- `ems/ems-backend/utils/sensorAggregation.js`
- `ems/ems-backend/controllers/sensorDataController.js`
- `ems/ems-backend/controllers/aiAnalyticsController.js`

Verification

```
curl -H "Authorization: Bearer $JWT" \
  "http://localhost:5000/api/sensor-data/dashboard-summary?deviceId=UUID&timeRange=30d"
# Query plan uses sensor_readings_hourly, not full sensor_readings scan
```

P-37 — No compression ✅ Fixed

Field	Detail
Severity	🟡 Medium
Category	Data lifecycle
Location	`ems/ems-backend/scripts/setup-timescaledb.sql`
Phase	3

What was the issue?

Raw sensor reading chunks consumed full disk footprint indefinitely until retention dropped them. JSON payloads compress well but Postgres default heap storage does not compress.

Why did it happen?

Standard Postgres row storage without columnar compression.

Impact

- ~70–90% more disk than necessary for data older than 7 days
- Larger backups and slower replication

Fix applied (production-grade)

TimescaleDB compression enabled on `sensor_readings` with policy: compress chunks older than **7 days**, segment by `"deviceId"`. Compressed chunks are read-only (automatic decompression on policy violation attempts).

Files changed

- `ems/ems-backend/scripts/setup-timescaledb.sql`

Verification

```
SELECT chunk_name, is_compressed
FROM timescaledb_information.chunks
WHERE hypertable_name = 'sensor_readings' AND is_compressed = true;
```

P-38 — No cold archival ✅ Fixed

Field	Detail
Severity	🟢 Low
Category	Data lifecycle
Location	`ems/ems-backend/scripts/archive-cold-data.js`
Phase	3

What was the issue?

After 90-day retention dropped raw chunks, no export existed for compliance or long-term analytics beyond hourly rollups still in Postgres.

Why did it happen?

Retention policy implemented before archival pipeline.

Impact

- Regulatory requirements for multi-year history unmet
- No offline backup of rollup data before retention

Fix applied (production-grade)

`scripts/archive-cold-data.js` exports `sensor_readings_hourly` rows older than N days to JSON files:

```
node scripts/archive-cold-data.js --days=90 --out=./archives
```

Output: `archives/hourly-rollups-before-YYYY-MM-DD.json` with `{ exportedAt, cutoff, rows }`. Schedule via cron before retention job or upload to S3 manually.

Files changed

- `ems/ems-backend/scripts/archive-cold-data.js`

Verification

```
cd ems/ems-backend && node scripts/archive-cold-data.js --days=90 --out=./archives
ls -la archives/
# JSON file with row count logged to stdout
```

Category J — Async Processing & Queues

P-39 — No message queue ✅ Fixed

Field	Detail
Severity	🟠 High
Category	Queue
Location	`ems/ems-backend/workers/jobQueues.js`
Phase	2

What was the issue?

All background work (ingest persist, anomaly detection, email, device deletion) ran synchronously on the Express event loop or as fire-and-forget promises without durability or retry.

Why did it happen?

No Redis/BullMQ infrastructure in initial deployment.

Impact

- Event loop blocked by email SMTP or heavy anomaly logic
- Lost work on process crash mid-operation
- No backpressure under spike load

Fix applied (production-grade)

`workers/jobQueues.js` implements four BullMQ queues when Redis available:

Queue	Worker	Concurrency	Purpose
`ingest`	Micro-batch flush	4 (configurable)	Persist readings (P-08)
`anomaly`	`runAnomalyCheck`	2	Threshold evaluation (P-41)
`email`	SMTP send	1, 5/sec limit	Alarm emails (P-40/P-55)
`device-delete`	`purgeDeviceData`	1	Async device purge (P-57)

`initAllQueues()` called from `server.js` on startup. Graceful inline fallback when Redis absent.

Files changed

- `ems/ems-backend/workers/jobQueues.js`
- `ems/ems-backend/server.js`

Verification

```
redis-cli KEYS 'bull:*'
curl http://localhost:5000/metrics | grep -E 'ingest_|anomaly_|emails_'
```

P-40 — Synchronous email sending ✅ Fixed

Field	Detail
Severity	🟠 High
Category	Queue
Location	`ems/ems-backend/workers/jobQueues.js` , `services/notificationService.js`
Phase	2

What was the issue?

Alarm notification emails sent via `await transporter.sendMail()` inline during anomaly handling. SMTP latency (200 ms–2 s) blocked the ingest/anomaly path.

Why did it happen?

Simplest integration with nodemailer in notification service.

Impact

- Ingest worker blocked on Gmail/SES response time
- SMTP failures propagated as ingest failures
- No retry on transient SMTP errors

Fix applied (production-grade)

`notificationService.js` calls `enqueueEmail()` which adds job to BullMQ `email` queue. Worker sends with **5 emails/sec** rate limit, **5 attempts** with exponential backoff. On success/failure, writes notification history via `logHistory()`. Falls back to inline send when queue unavailable.

Files changed

- `ems/ems-backend/workers/jobQueues.js`
- `ems/ems-backend/services/notificationService.js`

Verification

Trigger alarm; confirm ingest returns immediately; email arrives within seconds; `emails_sent_total` metric increments.

P-41 — Anomaly detection on event loop ✅ Fixed

Field	Detail
Severity	🟡 Medium
Category	Queue
Location	`ems/ems-backend/services/anomalyDetector.js`, `workers/jobQueues.js`, `services/ingestService.js`
Phase	2

What was the issue?

`checkAnomalies` ran as fire-and-forget on the main process after each ingest, still consuming CPU and DB connections on the API event loop.

Why did it happen?

Intermediate fix between fully synchronous and queued anomaly.

Impact

- CPU spikes on API process during alarm storms
- Anomaly DB queries compete with HTTP handlers

Fix applied (production-grade)

`ingestService.js` calls `enqueueAnomalyCheck(payload)` after persist. BullMQ `anomaly` worker runs `runAnomalyCheck()` with concurrency 2. Inline fallback with `.catch()` when Redis absent. Metric: `anomaly_checks_total`.

Files changed

- `ems/ems-backend/services/ingestService.js`
- `ems/ems-backend/services/anomalyDetector.js`
- `ems/ems-backend/workers/jobQueues.js`

Verification

Ingest breach payload; API response time unaffected; Redis queue `bull:anomaly:*` shows processed job.

Category K — Flutter Client

P-42 — SharedPreferences per call ✅ Fixed

Field	Detail
Severity	🟡 Medium
Category	Client
Location	`app/lib/services/cache_service.dart`, `app/lib/main.dart`, `app/lib/services/auth_service.dart`
Phase	1

What was the issue?

Multiple services called `SharedPreferences.getInstance()` independently on every auth/token read — async disk I/O repeated unnecessarily.

Why did it happen?

Each service obtained its own prefs reference without shared singleton.

Impact

- Slower app startup and login
- Redundant platform channel calls on Android

Fix applied (production-grade)

`CacheService` singleton with `init()` caching `SharedPreferences` instance. `main.dart` awaits `CacheService.instance.init()` before `runApp()`. `AuthService` uses `CacheService.instance.prefs` for token and refresh token keys.

Files changed

- `app/lib/services/cache_service.dart`
- `app/lib/main.dart`
- `app/lib/services/auth_service.dart`

Verification

```
cd app && flutter test test/unit/cache_service_test.dart
```

P-43 — Dashboard refetches summary per socket event ✅ Fixed

Field	Detail
Severity	🟠 High
Category	Client
Location	`app/lib/pages/dashboard/dashboard_page.dart`
Phase	1

What was the issue?

Dashboard registered an `AppState` listener that called `_loadSummary()` (full HTTP aggregate fetch) on every socket notification — defeating P-28/P-46 debouncing and causing HTTP storms.

Why did it happen?

Listener treated all state changes as requiring full summary refresh.

Impact

- 1 HTTP aggregate request per second per device during live ingest
- Backend overload from redundant dashboard-summary calls
- Battery drain on mobile

Fix applied (production-grade)

Removed blanket listener refetch. Summary loads only on: init, device/slave change, pull-to-refresh. Live values update via `AppState.liveReadings` from socket without HTTP.

Files changed

- `app/lib/pages/dashboard/dashboard_page.dart`

Verification

Open dashboard; ingest 10 readings; network inspector shows zero `dashboard-summary` calls during live updates.

P-44 — No pagination ✅ Fixed

Field	Detail
Severity	🟡 Medium
Category	Client
Location	`app/lib/pages/notifications_page.dart`, `alarm_history_page.dart`, `interval_history_page.dart`, `sensor_history_page.dart`, backend browse endpoints
Phase	1

What was the issue?

List screens loaded entire datasets in one request. Notification history, alarm history, interval history, and sensor browse could return thousands of rows.

Why did it happen?

Initial UI prototypes used simple `findMany` without page/limit.

Impact

- Slow first paint on history screens
- High memory use on low-end Android devices
- Timeouts on large orgs

Fix applied (production-grade)

Backend: `GET /sensor-data/readings?page=&limit=` via `getReadingsBrowse`; paginated alarm and interval history endpoints with `paginate()` helper (max limit 100 — P-52).

Flutter: Infinite scroll with `_page`, `_pageSize=30` on:

- `notifications_page.dart` — `getNotificationsPage()`
- `alarm_history_page.dart` — paginated alarm history API
- `interval_history_page.dart` — `getIntervalHistoryPage()`
- `sensor_history_page.dart` — paginated readings browse

Files changed

- `ems/ems-backend/controllers/sensorDataController.js`
- `ems/ems-backend/controllers/intervalHistoryController.js`
- `app/lib/pages/notifications_page.dart`
- `app/lib/pages/alarm_history_page.dart`
- `app/lib/pages/interval_history_page.dart`
- `app/lib/pages/sensor_history_page.dart`
- `app/lib/services/ems_api.dart`

Verification

Open notifications with 100+ items; initial load ≤30 items; scroll loads next page.

P-45 — No response compression ✅ Fixed

Field	Detail
Severity	🟡 Medium
Category	Client / Server
Location	`ems/ems-backend/server.js`

Field	Detail
Phase	1

What was the issue?

API responses (especially dashboard JSON with chart arrays) sent uncompressed over mobile networks.

Why did it happen?

Express default does not enable compression middleware.

Impact

- Larger payloads on 3G/4G connections
- Slower dashboard load times
- Higher bandwidth costs

Fix applied (production-grade)

`compression()` middleware registered early in `server.js` middleware stack (after helmet). Gzip/deflate applied to JSON responses above threshold automatically.

Files changed

- `ems/ems-backend/server.js`

Verification

```
curl -H "Accept-Encoding: gzip" -H "Authorization: Bearer $JWT" \
  --compressed -v "http://localhost:5000/api/sensor-data/dashboard-summary?deviceId=UUID&tim
# content-encoding: gzip
```

P-46 — Client renders every socket event ✅ Fixed

Field	Detail
Severity	🟡 Medium
Category	Client
Location	`app/lib/services/app_state.dart`
Phase	2

What was the issue?

Every `reading:new` socket event called `notifyListeners()` immediately, triggering full widget tree rebuilds at ingest frequency (up to 1 Hz × devices).

Why did it happen?

Direct notify on each socket callback without debounce.

Impact

- UI jank and elevated CPU on dashboard
- Battery drain from excessive repaints

Fix applied (production-grade)

`AppState._debouncedNotify()` uses 800 ms `Timer` — coalesces rapid reading updates to ~1 rebuild/sec max. Alarms and device switch events call immediate `notifyListeners()` for responsiveness.

Files changed

- `app/lib/services/app_state.dart`

Verification

Ingest 10 readings in 5 seconds; Flutter DevTools shows ≤2 rebuilds of dashboard metric widgets.

Category L — Security & Ops Hardening

P-47 — Single global ingest API key ✅ Fixed

Field	Detail
Severity	🟠 High
Category	Security
Location	`ems/ems-backend/utils/ingestAuth.js`, `controllers/deviceController.js`, `prisma/schema.prisma`
Phase	2

What was the issue?

All IoT gateways shared one `INGEST_API_KEY`. Compromise of any device credential exposed the entire ingest surface; per-device revocation impossible.

Why did it happen?

Simplest MVP auth — single shared secret in environment.

Impact

- Blast radius of key leak = all devices
- Cannot rate-limit or audit per device (blocks P-24)
- Key rotation requires updating every gateway simultaneously

Fix applied (production-grade)

- `Device.ingestApiKeyHash` column stores SHA-256 hash of per-device key
- `validateIngestKey(apiKey, deviceId)` accepts global fallback OR per-device hash via `crypto.timingSafeEqual`
- `createDevice` generates random 48-char hex key returned once in response
- `POST /devices/:id/regenerate-ingest-key` rotates key for compromised devices
- Enables per-device rate limiting (P-24)

Files changed

- `ems/ems-backend/utils/ingestAuth.js`
- `ems/ems-backend/controllers/deviceController.js`
- `ems/ems-backend/controllers/ingestController.js`
- `ems/ems-backend/prisma/schema.prisma`
- `ems/ems-backend/scripts/migrate-v3.1.sql`
- `ems/ems-backend/routes/devices.js`

Verification

```
# Create device; note ingestKey in response
curl -X POST http://localhost:5000/api/ingest \
  -H "x-api-key: DEVICE_SPECIFIC_KEY" -H "Content-Type: application/json" \
  -d '{"deviceId":"UUID","readings":[{"variableName":"PowerConsumption","value":1.0}]}'
# Wrong key for device → 401
```

P-48 — `applicationId` still `com.example.*` ✅ Fixed

Field	Detail
Severity	🟢 Low
Category	Release
Location	`app/android/app/build.gradle.kts`, `app/android/app/src/main/kotlin/com/smartagritech/ems/MainActivity.kt`
Phase	3

What was the issue?

Android `applicationId` and namespace remained default `com.example.*` placeholder, unsuitable for Play Store release and push notification certificates.

Why did it happen?

Flutter project scaffold defaults not updated before first release candidate.

Impact

- Play Store rejection or namespace collision
- Cannot publish production APK/AAB
- Package name mismatch with org branding

Fix applied (production-grade)

Set `namespace` and `applicationId` to **`com.smartagritech.ems`**. Moved `MainActivity.kt` to matching package path `com/smartagritech/ems/`.

Files changed

- `app/android/app/build.gradle.kts`
- `app/android/app/src/main/kotlin/com/smartagritech/ems/MainActivity.kt`

Verification

```
cd app && flutter build appbundle --release
# Verify applicationId in build output: com.smartagritech.ems
aapt dump badging build/app/outputs/apk/release/app-release.apk | grep package
```

P-49 — No JWT refresh ✅ Fixed

Field	Detail
Severity	🟡 Medium
Category	Security
Location	`ems/ems-backend/controllers/authController.js`, `app/lib/services/auth_service.dart`, `app/lib/services/api_client.dart`
Phase	3

What was the issue?

JWT access tokens had long or no explicit expiry strategy; mobile sessions either lasted too long (security risk) or expired abruptly forcing re-login. No refresh token rotation.

Why did it happen?

Initial auth used single long-lived JWT only.

Impact

- Users logged out mid-session when token expired
- Stolen JWT valid for extended period if no short expiry
- Poor mobile UX on backgrounded apps

Fix applied (production-grade)

Backend:

- Access token: **15 minutes** (`JWT_EXPIRES_IN=15m`)
- Refresh token: **30 days** (`JWT_REFRESH_DAYS=30`), stored hashed in `RefreshToken` model
- `POST /auth/refresh` — validates refresh token, rotates (delete old, issue new pair)
- `POST /auth/logout` — deletes refresh token hash

Flutter:

- `AuthService` stores refresh token in `CacheService`
- `ApiClient.onRefreshToken` calls `/auth/refresh` on 401, retries original request once
- Silent renew without user interaction

Files changed

- `ems/ems-backend/controllers/authController.js`
- `ems/ems-backend/routes/auth.js`
- `ems/ems-backend/prisma/schema.prisma`

- `ems/ems-backend/scripts/migrate-v3.1.sql`
- `app/lib/services/auth_service.dart`
- `app/lib/services/api_client.dart`

Verification

Wait 16 min with app open; perform API action; network log shows `/auth/refresh` then successful retry without login screen.

P-50 — No observability ✅ Fixed

Field	Detail
Severity	🟡 Medium
Category	Ops
Location	`ems/ems-backend/utils/metrics.js`, `utils/logger.js`, `server.js`
Phase	3

What was the issue?

No structured logging, metrics endpoint, or ingest/error counters. Production incidents required tailing unstructured console output with no aggregation path.

Why did it happen?

Observability deferred until core features stabilised.

Impact

- Cannot alert on ingest failure rate
- No request volume visibility
- Difficult post-incident analysis

Fix applied (production-grade)

Metrics: `utils/metrics.js` in-process counters exposed at `GET /metrics` (Prometheus text format):

- `ingest_total`, `ingest_errors_total`, `ingest_queued_total`
- `anomaly_checks_total`, `emails_sent_total`, `emails_failed_total`
- `http_requests_total{method,route,status}`

Logging: `utils/logger.js` — JSON structured logs in production, readable format in development. Used across queues, flush service, and server startup.

HTTP middleware: Increments `http_requests_total` on response finish.

Files changed

- `ems/ems-backend/utils/metrics.js`
- `ems/ems-backend/utils/logger.js`
- `ems/ems-backend/server.js`
- `ems/ems-backend/workers/jobQueues.js`
- `ems/ems-backend/controllers/ingestController.js`

Verification

```
curl http://localhost:5000/metrics
curl http://localhost:5000/health
# Ingest payload; metrics show ingest_total increment
```

Category M — Additional Findings

P-51 — Device list N+1 summary calls ✅ Fixed

Field	Detail
Severity	🔴 Critical
Category	Client / API
Location	`app/lib/pages/devices/devices_page.dart`, `ems/ems-backend/controllers/deviceController.js`
Phase	1

What was the issue?

Devices page called `getDashboardSummary()` per device to show metric cards — N expensive aggregate HTTP requests for N devices on every page load.

Why did it happen?

Dashboard summary endpoint reused for device list preview without batch API.

Impact

- 50 devices = 50 aggregate queries on page open
- Multi-second devices page load

- Backend overload proportional to device count

Fix applied (production-grade)

Client: `devices_page.dart` uses `getDevicesForUi(withMetrics: false)` — no per-device summary fan-out.

Backend: `GET /devices?withMetrics=true` attaches `latestMetrics` from Redis hash `device:{id}:latest` via `attachLatestMetrics()` — O(1) Redis reads per device, no SQL aggregation.

Files changed

- `app/lib/pages/devices/devices_page.dart`
- `app/lib/services/ems_api.dart`
- `ems/ems-backend/controllers/deviceController.js`

Verification

Open devices page with 20 devices; network tab shows one `GET /devices` call, zero `dashboard-summary` calls.

P-52 — `paginate` no max limit ✅ Fixed

Field	Detail
Severity	🟠 High
Category	API
Location	`ems/ems-backend/utils/helpers.js`
Phase	1

What was the issue?

Clients could request `?limit=100000`, forcing full table scans and multi-MB JSON responses.

Why did it happen?

`paginate()` passed query limit directly to Prisma without cap.

Impact

- DoS vector via single authenticated request
- OOM on server or client from huge responses

Fix applied (production-grade)

`paginate()` clamps: `limit = Math.min(100, Math.max(1, parseInt(query.limit) || 20))`.

Files changed

- `ems/ems-backend/utils/helpers.js`

Verification

```
curl -H "Authorization: Bearer $JWT" "http://localhost:5000/api/devices?limit=9999"
# Returns at most 100 items
```

P-53 — Two divergent `paginate` impls ✅ Fixed

Field	Detail
Severity	🟢 Low
Category	API
Location	`ems/ems-backend/utils/pagination.js`, `utils/helpers.js`
Phase	1

What was the issue?

Two separate pagination helper functions existed with slightly different defaults and behaviour, causing inconsistent API responses across controllers.

Why did it happen?

Helper extracted to new file without removing original.

Impact

- Some endpoints capped at 100, others unbounded (before P-52)
- Maintenance confusion

Fix applied (production-grade)

`utils/pagination.js` re-exports `helpers.paginate` as single source of truth. All controllers import from one path.

Files changed

- `ems/ems-backend/utils/pagination.js`
- `ems/ems-backend/utils/helpers.js`

Verification

```
grep -r "paginate" ems/ems-backend/controllers --include="*.js"
# All imports resolve to same implementation
```

P-54 — Prisma errors not mapped ✅ Fixed

Field	Detail
Severity	🟡 Medium
Category	API
Location	`ems/ems-backend/middleware/errorHandler.js`
Phase	1

What was the issue?

Prisma constraint violations returned generic 500 errors with internal codes exposed or unhelpful messages. Clients could not distinguish duplicate key vs not found.

Why did it happen?

Default Express error handler without Prisma code mapping.

Impact

- Poor API consumer experience
- Difficult client-side error handling
- Internal error details leaked in some cases

Fix applied (production-grade)

`errorHandler.js` maps Prisma codes:

Code	HTTP	Meaning
P2002	409	Unique constraint violation
P2025	404	Record not found

Code	HTTP	Meaning
P2003, P2014	400	Foreign key / relation violation

Files changed

- `ems/ems-backend/middleware/errorHandler.js`

Verification

Create duplicate email user → 409. Update non-existent ID → 404.

P-55 — Email: Gmail, no pooling ✅ Fixed

Field	Detail
Severity	🟠 High
Category	Infrastructure
Location	`ems/ems-backend/config/nodemailer.js` , `workers/jobQueues.js`
Phase	2

What was the issue?

Nodemailer used default Gmail service without connection pooling. Each email opened new SMTP connection — slow and hit Gmail connection limits during alarm storms.

Why did it happen?

Development Gmail credentials used without production SMTP configuration.

Impact

- SMTP connection errors under burst alarm load
- 2–5 s per email without pool
- Gmail daily send limits hit quickly

Fix applied (production-grade)

Transporter: `nodemailer.createTransport` with `pool: true` , `maxConnections: 3` (SES) or `2` (Gmail). Supports `SMTP_HOST/PORT/USER/PASS` for SES/SendGrid override via env.

Queue: Email worker rate-limited to 5/sec with retry backoff (P-40).

Files changed

- `ems/ems-backend/config/nodemailer.js`
- `ems/ems-backend/workers/jobQueues.js`
- `ems/ems-backend/.env.example`

Verification

Trigger 10 alarms rapidly; all emails delivered; no SMTP connection errors in logs; `emails_sent_total` = 10.

P-56 — `createDevice` N+1 inserts ✅ Fixed

Field	Detail
Severity	🟡 Medium
Category	Write
Location	`ems/ems-backend/controllers/deviceController.js`
Phase	1

What was the issue?

Creating a device from template inserted config variables one-by-one in a loop — one INSERT per template variable per slave.

Why did it happen?

ORM loop pattern mirroring template structure iteration.

Impact

- Slow device provisioning (30+ inserts for typical template)
- Long transaction holding locks during onboarding

Fix applied (production-grade)

After creating each `deviceConfigSlave`, collect all variable rows and call `deviceConfigVariable.createMany({ data: rows })` in single batch per slave.

Files changed

- `ems/ems-backend/controllers/deviceController.js`

Verification

Create device from template with 20 variables; Prisma log shows one `createMany` per slave, not 20 individual creates.

P-57 — `deleteDevice` synchronous mass delete ✅ Fixed

Field	Detail
Severity	🟡 Medium
Category	Write
Location	`ems/ems-backend/controllers/deviceController.js`, `workers/jobQueues.js`
Phase	2

What was the issue?

Deleting a device synchronously deleted all related rows including potentially millions of `sensor_readings` in one HTTP request — timeout and lock risk.

Why did it happen?

Cascade delete in single controller method without batching or async job.

Impact

- HTTP timeout on devices with large history
- Long-running transaction blocking ingest for same org
- Gateway 504 errors to admin UI

Fix applied (production-grade)

When BullMQ available: `deleteDevice` enqueues `device-delete` job and returns **202** `{ success: true, queued: true, deviceId }`. Worker `purgeDeviceData()` deletes in batches of 5000 readings, then cascades config, alarms, schedules, commands, and device row. Sync fallback `purgeDeviceSync()` when Redis absent.

Files changed

- `ems/ems-backend/controllers/deviceController.js`
- `ems/ems-backend/workers/jobQueues.js`

Verification

```
curl -X DELETE -H "Authorization: Bearer $JWT" http://localhost:5000/api/devices/UUID
# Response: 202 with queued:true when Redis enabled
# Device eventually removed; logs show "device deleted async"
```

P-58 — No helmet / body-size limit ✅ Fixed

Field	Detail
Severity	🟡 Medium
Category	Security
Location	`ems/ems-backend/server.js`
Phase	1

What was the issue?

Express app lacked security headers and JSON body size limit. Large malicious payloads could exhaust memory; missing headers increased XSS/clickjacking surface.

Why did it happen?

Default Express setup without hardening middleware.

Impact

- Potential DoS via large JSON bodies
- Missing standard security headers in production

Fix applied (production-grade)

- `helmet()` — standard security headers
- `express.json({ limit: '256kb' })` and `urlencoded({ limit: '256kb' })`

Ingest payloads with many variables remain under 256 KB at normal gateway message sizes.

Files changed

- `ems/ems-backend/server.js`

Verification

```
curl -X POST http://localhost:5000/api/auth/login \
  -H "Content-Type: application/json" -d "$(python -c 'print("{\"email\":\"x\",\"password\":'
# Response: 413 Payload Too Large
curl -I http://localhost:5000/health  # Security headers present
```

P-59 — Device command no actuation ack ✅ Fixed

Field	Detail
Severity	🟡 Medium
Category	IoT
Location	`DeviceCommand` model, `controllers/deviceController.js`, `controllers/ingestController.js`
Phase	3

What was the issue?

UI could toggle device switch state in Postgres immediately without gateway confirmation. No command lifecycle — UI showed ON while physical device remained OFF.

Why did it happen?

Switch state updated directly via CRUD without IoT command/ack pattern.

Impact

- Incorrect switch state displayed to operators
- No audit trail of commanded vs acknowledged actions
- Scheduled tasks and manual toggles indistinguishable

Fix applied (production-grade)

Model: `DeviceCommand` with statuses `PENDING`, `ACKNOWLEDGED`, `FAILED`, `TIMEOUT`.

API:

- `PATCH /api/devices/:id/switch` — creates PENDING command, emits `device:command` to `device_{id}` room, 30 s timeout watchdog
- `POST /api/ingest/command-ack` — gateway acknowledges with `deviceId`, `commandId`, `status`; updates device `switchState` on ACK
- `GET /api/devices/:id/commands/:commandId` — poll command status

Flutter: Can listen for `device:command` socket events to reflect real state.

Files changed

- `ems/ems-backend/prisma/schema.prisma`
- `ems/ems-backend/scripts/migrate-v3.1.sql`
- `ems/ems-backend/controllers/deviceController.js`
- `ems/ems-backend/controllers/ingestController.js`
- `ems/ems-backend/routes/devices.js`
- `ems/ems-backend/routes/ingest.js`

Verification

```
# Admin toggles switch
curl -X PATCH -H "Authorization: Bearer $JWT" -H "Content-Type: application/json" \
  -d '{"action":"ON"}' http://localhost:5000/api/devices/UUID/switch
# Gateway acks
curl -X POST http://localhost:5000/api/ingest/command-ack \
  -H "x-api-key: $DEVICE_KEY" -H "Content-Type: application/json" \
  -d '{"deviceId":"UUID","commandId":"CMD_ID","status":"ACKNOWLEDGED"}'
# Device switchState = ON; command status ACKNOWLEDGED
```

P-60 — Missing index on `device_users` ✅ Fixed

Field	Detail
Severity	🟡 Medium
Category	Read
Location	`ems/ems-backend/prisma/schema.prisma`
Phase	1

What was the issue?

USER-role device list query filters by `deviceUsers.some({ userId })` without composite index. Every device list request for field users scanned the `device_users` join table.

Why did it happen?

Join table added without index aligned to access pattern.

Impact

- Slow device list for USER role as org device count grows

- Sequential scan on `device_users` for every app open

Fix applied (production-grade)

Added `@@index([userId, deviceId])` on `DeviceUser` model in Prisma schema.

Files changed

- `ems/ems-backend/prisma/schema.prisma`

Verification

```
EXPLAIN ANALYZE SELECT d.* FROM devices d
  JOIN device_users du ON du."deviceId" = d.id
  WHERE du."userId" = 'UUID';
-- Index Scan using device_users_userId_deviceId_idx
```

---

17. Phased Rollout Plan

All four phases are ✅ **Complete** as of v3.1 (documented in v4.0).

Phase 0 — Correctness ✅ Complete

Items: P-01, P-02, P-03, P-04, P-05, P-15, P-16, P-17

Client build fixes, socket contract, HTTP timeouts, anomaly N+1 and cooldown.

Phase 1 — Stop the bleeding ✅ Complete

Items: P-06, P-07, P-11, P-12, P-13, P-19, P-21, P-25, P-26, P-42, P-43, P-44, P-45, P-51, P-52, P-53, P-54, P-56, P-58, P-60

Bulk ingest SQL, remove log amplification, SQL aggregates, indexes, auth cache, rate limits, pagination, error mapping.

Phase 2 — Decouple & scale out ✅ Complete

Items: P-08, P-09, P-10, P-18, P-20, P-22, P-23, P-24, P-27, P-28, P-29, P-39, P-40, P-41, P-46, P-47, P-55, P-57

Redis, BullMQ (all queues), Socket cluster adapter, per-device keys and rate limits, reference cache, email pool, async delete.

Phase 3 — Long-term scale ✅ Complete

Items: P-14, P-30, P-31, P-32, P-33, P-34, P-35, P-36, P-37, P-38, P-48, P-49, P-50, P-59

TimescaleDB, PgBouncer, read replica, narrow value table, retention/compression/archival, JWT refresh, observability, actuation ack, Play Store applicationId.

---

18. Problem Index

ID	Sev	Area	Title	Status
P-01	🔴	Client	Missing INTERNET permission	✅ Fixed
P-02	🔴	Client	Socket event name mismatch	✅ Fixed
P-03	🔴	Build	Core library desugaring	✅ Fixed
P-04	🟠	Client	No HTTP timeout	✅ Fixed
P-05	🟢	Client	Dead kIsWeb ternary	✅ Fixed
P-06	🔴	Write	N+1 writes per ingest	✅ Fixed
P-07	🔴	Write	deviceConfigVariableLog amplification	✅ Fixed
P-08	🟠	Write	No write batching	✅ Fixed
P-09	🟠	Write	currentValue every tick	✅ Fixed
P-10	🟡	Write	COPY bulk insert	✅ Fixed
P-11	🔴	Read	Dashboard summary in RAM	✅ Fixed
P-12	🟠	Read	AI analytics in RAM	✅ Fixed
P-13	🔴	Read	Missing DB indexes	✅ Fixed
P-14	🟡	Read	JSON not indexable	✅ Fixed
P-15	🟠	N+1	Anomaly findUnique loop	✅ Fixed
P-16	🟠	N+1	Anomaly no cooldown	✅ Fixed
P-17	🟡	N+1	Notification contacts loop	✅ Fixed
P-18	🟡	N+1	Socket auth DB hit	✅ Fixed
P-19	🟠	N+1	protect DB hit	✅ Fixed
P-20	🟠	Cache	No caching layer	✅ Fixed
P-21	🟠	Cache	Aggregates not cached	✅ Fixed
P-22	🟡	Cache	Reference data	✅ Fixed

ID	Sev	Area	Title	Status
P-23	🟠	Limit	Cluster rate limits	✅ Fixed
P-24	🟠	Limit	Ingest per-IP	✅ Fixed
P-25	🟡	Limit	API limit too tight	✅ Fixed
P-26	🟠	Limit	Auth limits	✅ Fixed
P-27	🟠	Socket	Redis adapter	✅ Fixed
P-28	🟡	Socket	Emit throttling	✅ Fixed
P-29	🟡	Socket	Org-wide broadcast	✅ Fixed
P-30	🟠	DB	PgBouncer	✅ Fixed
P-31	🟡	DB	Prisma pool tune	✅ Fixed
P-32	🟡	DB	Read replicas	✅ Fixed
P-33	🟠	DB	TimescaleDB	✅ Fixed
P-34	🟡	DB	Partitioning	✅ Fixed
P-35	🟠	Data	Retention	✅ Fixed
P-36	🟡	Data	Downsampling	✅ Fixed
P-37	🟡	Data	Compression	✅ Fixed
P-38	🟢	Data	Cold archival	✅ Fixed
P-39	🟠	Queue	Message queue	✅ Fixed
P-40	🟠	Queue	Sync email	✅ Fixed
P-41	🟡	Queue	Anomaly on event loop	✅ Fixed
P-42	🟡	Client	SharedPreferences per call	✅ Fixed
P-43	🟠	Client	Dashboard socket refetch	✅ Fixed
P-44	🟡	Client	Pagination	✅ Fixed
P-45	🟡	Client	Response compression	✅ Fixed
P-46	🟡	Client	Socket UI storm	✅ Fixed
P-47	🟠	Security	Global ingest key	✅ Fixed
P-48	🟢	Release	applicationId placeholder	✅ Fixed
P-49	🟡	Security	JWT refresh	✅ Fixed
P-50	🟡	Ops	Observability	✅ Fixed
P-51	🔴	Client	Device list N+1	✅ Fixed

ID	Sev	Area	Title	Status
P-52	🔴	API	paginate no cap	✅ Fixed
P-53	🟢	API	Duplicate paginate	✅ Fixed
P-54	🟡	API	Prisma error mapping	✅ Fixed
P-55	🔴	Infra	Email scaling	✅ Fixed
P-56	🟡	Write	createDevice N+1	✅ Fixed
P-57	🟡	Write	deleteDevice mass delete	✅ Fixed
P-58	🟡	Security	helmet / body limit	✅ Fixed
P-59	🟡	IoT	No actuation ack	✅ Fixed
P-60	🟡	Read	device_users index	✅ Fixed

---

19. Production Verification Checklist

Run after deploy, infrastructure change, or v3.1 migration.

1. Health & metrics

```
curl http://localhost:5000/health
curl http://localhost:5000/metrics | head -30
```

2. Database & TimescaleDB

```
psql $DATABASE_URL -c "SELECT extversion FROM pg_extension WHERE extname='timescaledb';"
psql $DATABASE_URL -c "SELECT hypertable_name FROM timescaledb_information.hypertables;"
psql $DATABASE_URL -c "SELECT * FROM timescaledb_information.jobs WHERE proc_name LIKE '%ret
```

3. v3.1 schema migration

```
psql $DATABASE_URL -f ems/ems-backend/scripts/migrate-v3.1.sql
# Verify new tables/columns:
psql $DATABASE_URL -c "\d refresh_tokens"
psql $DATABASE_URL -c "\d device_commands"
psql $DATABASE_URL -c "\d sensor_reading_values"
psql $DATABASE_URL -c "SELECT column_name FROM information_schema.columns WHERE table_name='
```

4. Redis & queues (when enabled)

```
redis-cli ping
redis-cli KEYS 'bull:*' | head
# After ingest with REDIS_URL:
curl http://localhost:5000/metrics | grep ingest_
```

5. Ingest — sync and queued modes

```
# Per-device key (or global INGEST_API_KEY fallback)
curl -X POST http://localhost:5000/api/ingest \
  -H "x-api-key: $INGEST_API_KEY" -H "Content-Type: application/json" \
  -d '{"deviceId":"UUID","readings":[{"variableName":"PowerConsumption","value":1.2}]}'

# With Redis: {"success":true,"queued":true}
# Without Redis: {"success":true}

redis-cli HGETALL device:UUID:latest
```

6. Read path & caching

```
curl -H "Authorization: Bearer $JWT" \
  "http://localhost:5000/api/sensor-data/dashboard-summary?deviceId=UUID&timeRange=24h"

curl -H "Authorization: Bearer $JWT" \
  "http://localhost:5000/api/sensor-data/latest?deviceId=UUID"

curl -H "Authorization: Bearer $JWT" \
  "http://localhost:5000/api/devices?withMetrics=true"
```

7. Auth refresh

```
# Login → save refreshToken
curl -X POST http://localhost:5000/api/auth/login \
  -H "Content-Type: application/json" -d '{"email":"user@org.com","password":"***"}'

curl -X POST http://localhost:5000/api/auth/refresh \
  -H "Content-Type: application/json" -d '{"refreshToken":"..."}'
```

8. Device actuation (P-59)

```
curl -X PATCH -H "Authorization: Bearer $JWT" -H "Content-Type: application/json" \
  -d '{"action":"ON"}' http://localhost:5000/api/devices/UUID/switch

curl -X POST http://localhost:5000/api/ingest/command-ack \
  -H "x-api-key: $DEVICE_KEY" -H "Content-Type: application/json" \
  -d '{"deviceId":"UUID","commandId":"CMD_ID","status":"ACKNOWLEDGED"}'
```

9. Async device delete (P-57)

```
curl -X DELETE -H "Authorization: Bearer $JWT" http://localhost:5000/api/devices/UUID
# Expect 202 { queued: true } when Redis enabled
```

10. Cold archival (P-38)

```
cd ems/ems-backend && node scripts/archive-cold-data.js --days=90 --out=./archives
```

11. Flutter client

```
cd app && flutter analyze
cd app && flutter test
# Verify applicationId
grep applicationId app/android/app/build.gradle.kts
```

Key files reference

Area	Path
Ingest pipeline	`ems/ems-backend/services/ingestService.js`, `workers/jobQueues.js`
Redis latest + flush	`services/valueFlushService.js`, `config/redis.js`
SQL aggregation	`utils/sensorAggregation.js`
Narrow value table	`scripts/migrate-v3.1.sql`, `services/ingestService.js`
Response q/ reference cache	`utils/responseCache.js`, `utils/referenceCache.js`
Auth + refresh	`controllers/authController.js`, `utils/ingestAuth.js`
Rate limits	`middleware/rateLimiter.js`
Device commands	`controllers/deviceController.js`, `controllers/ingestController.js`
TimescaleDB setup	`scripts/setup-timescaledb.sql`

Area	Path
PgBouncer	`scripts/pgbouncer.ini`
Observability	`utils/metrics.js` , `utils/logger.js`
Flutter API + auth	`app/lib/services/ems_api.dart` , `api_client.dart` , `auth_service.dart`

---

*SmartAgriTech EMS Optimization Guide **v4.0** · June 2026 · **60/60 problems fixed***